Table of Contents

Kotlin Introduction **1**

Jetpack Compose for UI/UX in Kotlin **1**

Getting Started with Compose 1
Formatting Basics 5
Sizing 5
Modifier Objects 5
Compose: Displaying Images 8

Library Import Guidelines **10**

Enabling Automatic Library Imports 10
Do not delete or overwrite the namespace in Kotlin 11
Use the Composable Import Whenever Possible 11
Leave Unresolved Objects Alone 12
Do Not Create Anything 12
Sometimes Trial and Error is Necessary 12

Compose: Text Inputs and Buttons **13**

Handling State with an Event Listener (Callback function) **20**

Kotlin Introduction

Kotlin is officially supported by Google and is currently recognized as the most common industry-standard programming language for native Android development.
https://kotlinlang.org/

Since being introduced in 2017. Kotlin has largely replaced Java for Android development because it offers modern language features, safer code, and full compatibility with existing Java code.

Cross platform development with frameworks like Flutter can be enormously beneficial. However sometimes native development is preferred when efficient memory use or high performance are critical considerations for an app. Also, native Kotlin provides day 0 access to new features and updates when Google releases them for Android. Sometimes working with features such as the camera, audio and more can be easier done with native code.

Jetpack Compose for UI/UX in Kotlin

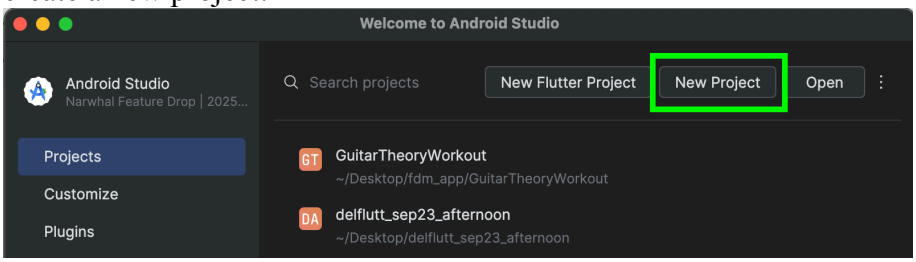
Jetpack Compose is a modern, flexible framework that has recently become the preferred approach for building UI/UX in Kotlin. It addresses many limitations of the older XML-based layout system, and the Kotlin team now recommends using Jetpack Compose for new projects.

Decent simple documentation on Jetpack compose controls can be found at:
https://composables.com/material3/outlinedtextfield

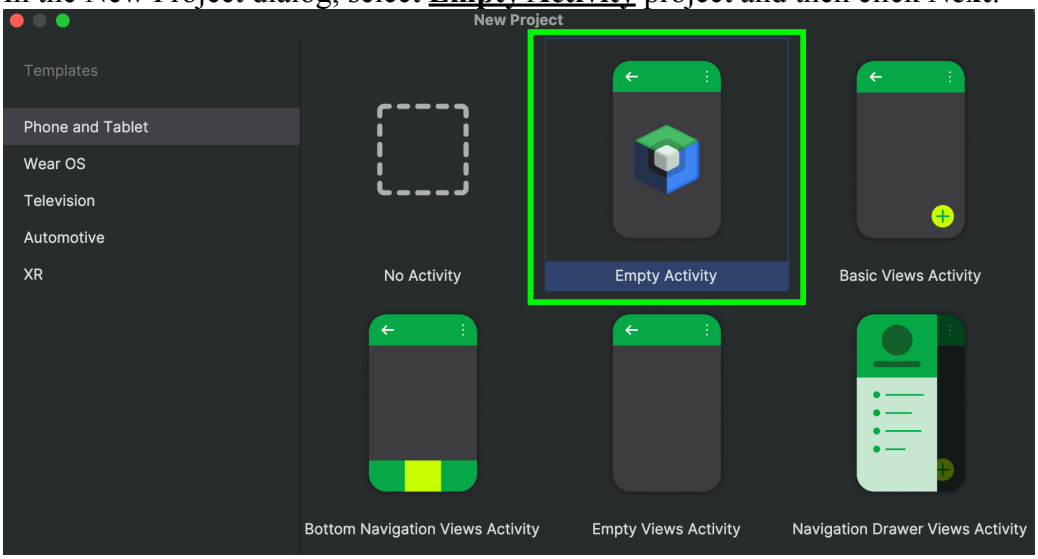
Getting Started with Compose

Example 1: Working with Compose

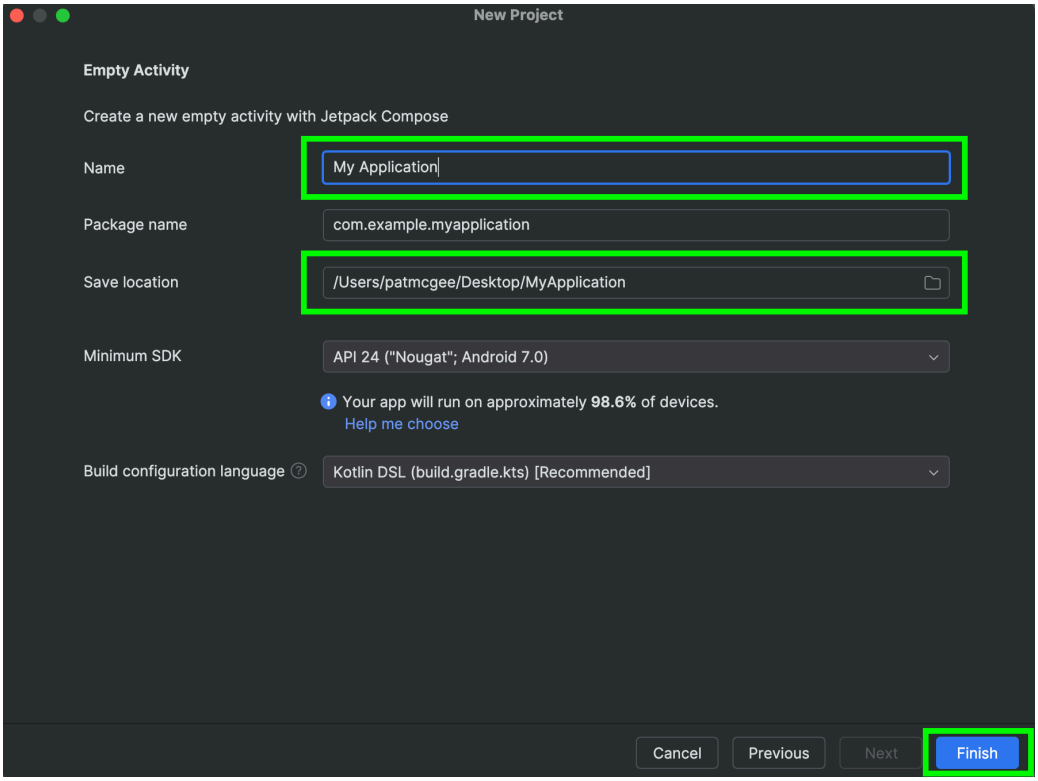
This example walks through some layout creation basics with Jetpack Compose. To do this, create a new project:



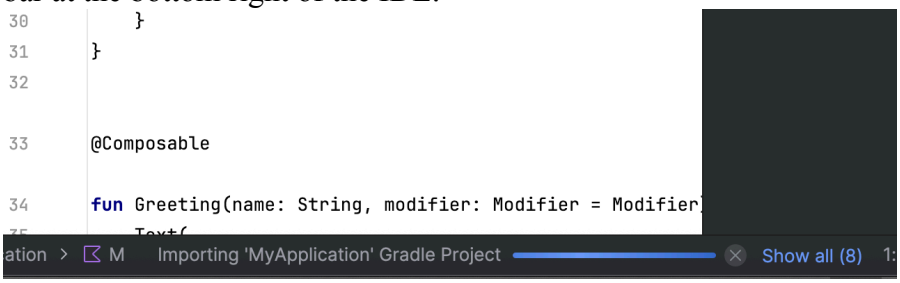
In the New Project dialog, select **Empty Activity** project and then click **Next**.



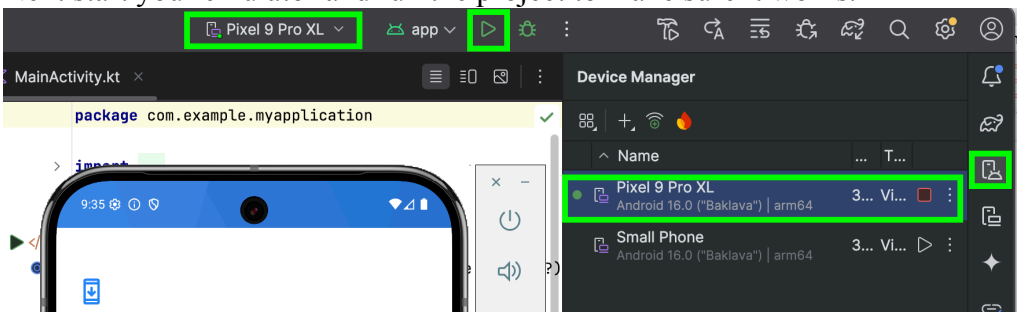
In the dialog that follows, leave the default settings but you may change the project name and location. When done, click the **Finish** button.



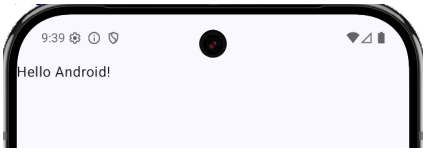
Clicking Finish triggers activity to initialize the project. Setup tasks are indicated by the progress bar at the bottom right of the IDE.



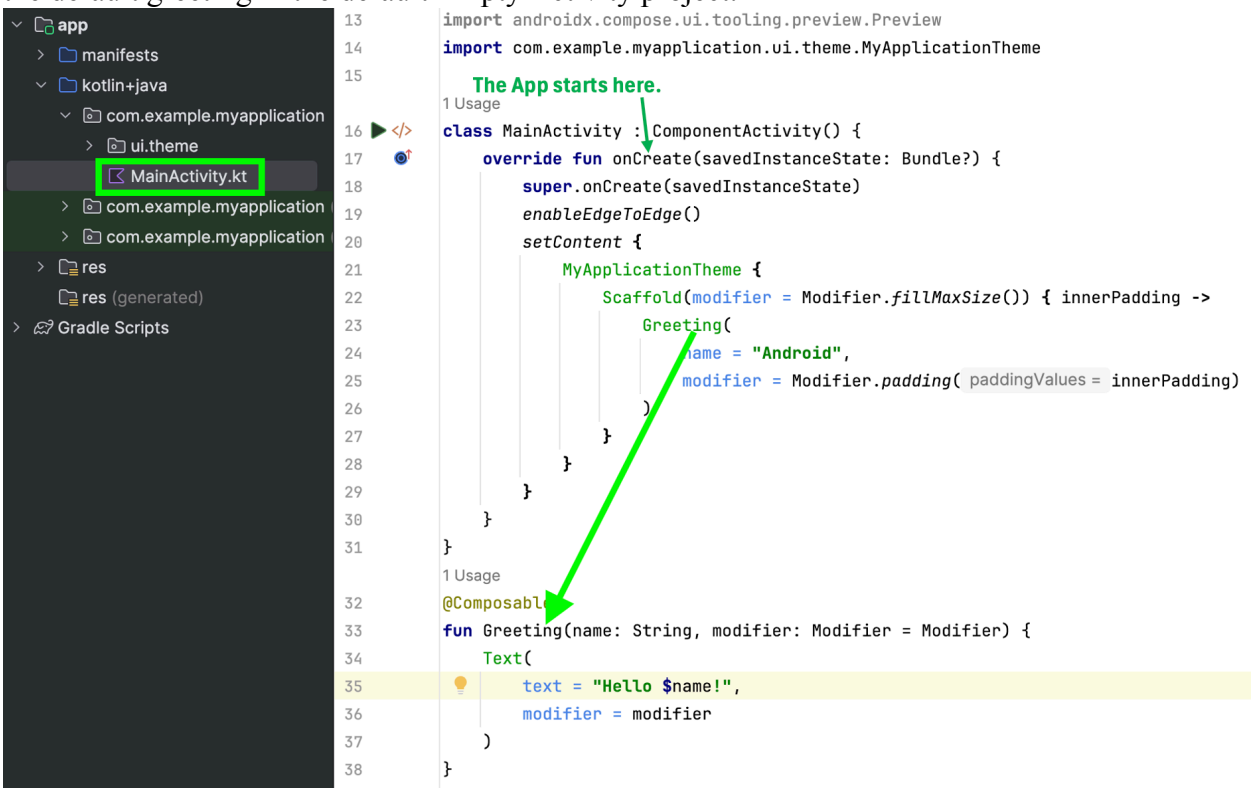
Next start your emulator and run the project to make sure it works.



Ideally you will see the following output in the emulator if successful.



Notice how a composable Text control object is already included in **MainActivity.kt** to display the default greeting in the default Empty Activity project.



Exercise 1 (1 mark)

What call back function is implemented in the MainActivity class to trigger other tasks to set up the main screen?

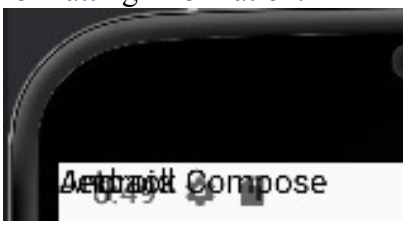
```
onCreate()
```

Example 2: Modifying the onCreate handler and screen output

This next step involves making changes to show a name of a person and a message on the main app screen. To build this example, replace the MainActity

```
class MainActivity : ComponentActivity() {
    override fun onCreate(savedInstanceState: Bundle?) {
        super.onCreate(savedInstanceState)
        enableEdgeToEdge()
        setContent {
            MessageCard(author = "Pat", body = "Jetpack Compose")
        }
    }
}
// @Composable tells Kotlin compiler that function is a piece of Jetpack Compose UI.
@Composable
fun MessageCard(author: String, body: String) {
    Text(text = author)
    Text(text = body)
}
```

When running the code as is, the sender's name and message overlap each other without any formatting information.



Formatting Basics

To enable basic formatting, this section presents some helpful structures:

Row and Column Functions

Column and Row objects help to implement layout arrangements in Kotlin.

Sizing

For sizing, the following units are recommended:

- **dp** (density-independent pixels) are for layout sizes, margins, and padding.
- **sp** (scale-independent pixels) are for font sizes.

Modifier Objects

Modifier object can be used to add spacing and padding for different controls:

```
Spacer(modifier=modifier.height(40.dp))
Text(text=author, fontSize=24.sp, modifier=modifier.padding(20.dp))
```

Example 3: Using a Column Function

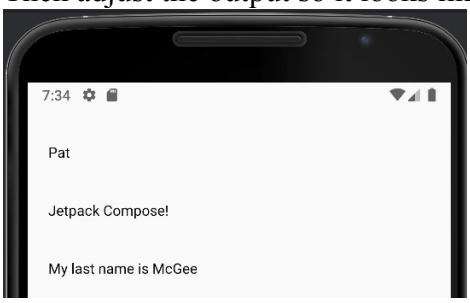
To fix the layout problem in Example 2, a Column function can be used to organize controls vertically within it. A modifier object can be used to help with spacing where dimensions are specified in **dp** units. Font sizes can be controlled with **sp** units.

To build this example, replace the MessageCard function in Example 2 with this version.

```
@Composable
fun MessageCard(author: String, body: String) {
    val modifier: Modifier = Modifier
    Column {
        Spacer(modifier = modifier.height(40.dp))
        Text(
            text = author,
            fontSize = 24.sp,
            modifier = modifier.padding(20.dp)
        )
        Text(
            text = body,
            fontSize = 16.sp
        )
    }
}
```

Exercise 2 (2 marks)

Then adjust the output so it looks like the following but it needs to show your code:

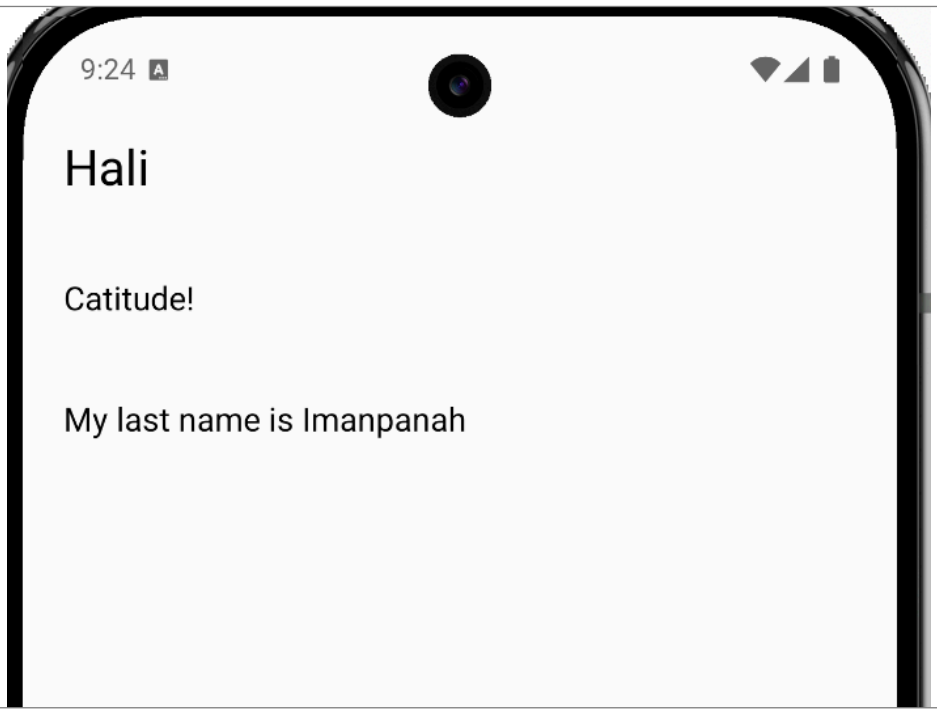


Show your MainActivity code as text after the changes here (no screenshots please):

```
class MainActivity : ComponentActivity() {
    override fun onCreate(savedInstanceState: Bundle?) {
        super.onCreate(savedInstanceState)
        enableEdgeToEdge()
```

```
        setContent {
            MessageCard(author = "Hali", body = "Catitude", lastName =
"Imanpanah")
        }
    }
}
// @Composable tells Kotlin compiler that function is a piece of Jetpack
Compose UI.
@Composable
fun MessageCard(author: String, body: String, lastName: String) {
    val modifier: Modifier = Modifier
    Column (modifier = modifier.padding(20.dp)){
        Spacer(modifier = modifier.height(40.dp))
        Text(
            text = author,
            fontSize = 24.sp,
        )
        Spacer(modifier = modifier.height(40.dp))
        Text(
            text = "$body!",
            fontSize = 16.sp
        )
        Spacer(modifier = modifier.height(40.dp))

        Text(
            text = "My last name is $lastName",
            fontSize = 16.sp
        )
    }
}
```

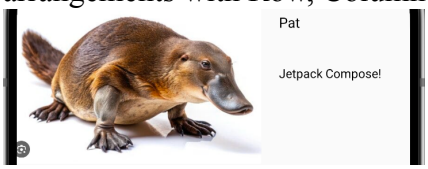
Show the screenshot of output after making this change but show your first name and last name instead:



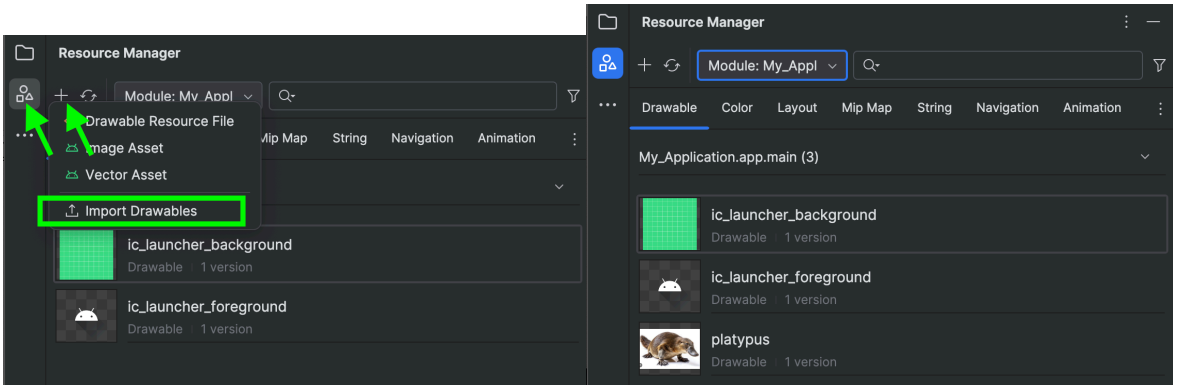
Compose: Displaying Images

Example 4: Displaying Images

This example shows how to display images with Kotlin in addition to demonstrating formatting arrangements with Row, Column and Modifier objects.



To add images, click on the **Resource Manager** icon at the top left of Android Studio. Then click the plus sign and select **Import Drawables**. From there you will be provided a dialog which will allow you to navigate to the image. After selecting the image and accepting all default settings, the image will then appear in the **Drawable** list.



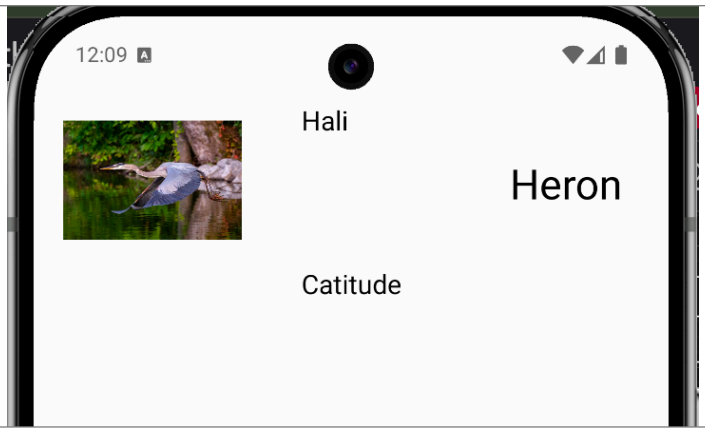
Starting with Example 3, replace the MessageCard function with the following revision:

```
@Composable
fun MessageCard(author: String, body:String) {
    // Objects declared with 'val' can't be reassigned.
    val modifier: Modifier = Modifier
    // Creates 1 row which has two columns of vertically organized controls.
    Row(modifier = Modifier.fillMaxWidth()) {
        Column(modifier = Modifier.weight(1f)) {
            Spacer(modifier = modifier.height(50.dp))
            Image(
                painter = painterResource(R.drawable.platypus),
                contentDescription = "Contact profile picture",
                modifier = Modifier.height(80.dp) // constrain image
            )
        }
        Column( ) {
            // Adds space above first control.
            Spacer(modifier = modifier.height(40.dp))
            Text(text = author,  fontSize = 16.sp,
                modifier = modifier.padding(20.dp))
            Text(text = body,  modifier.padding(20.dp))
        }
    }
}
```

Exercise 3 (2 marks)

Starting with the code from the last step, replace the image and create a third column which shows an animal's name which starts with the first letter of your name. The animal name should use a larger font. Change the first name to your first name. The output should look something like the following but with your first name.

Show a screenshot of the output here.



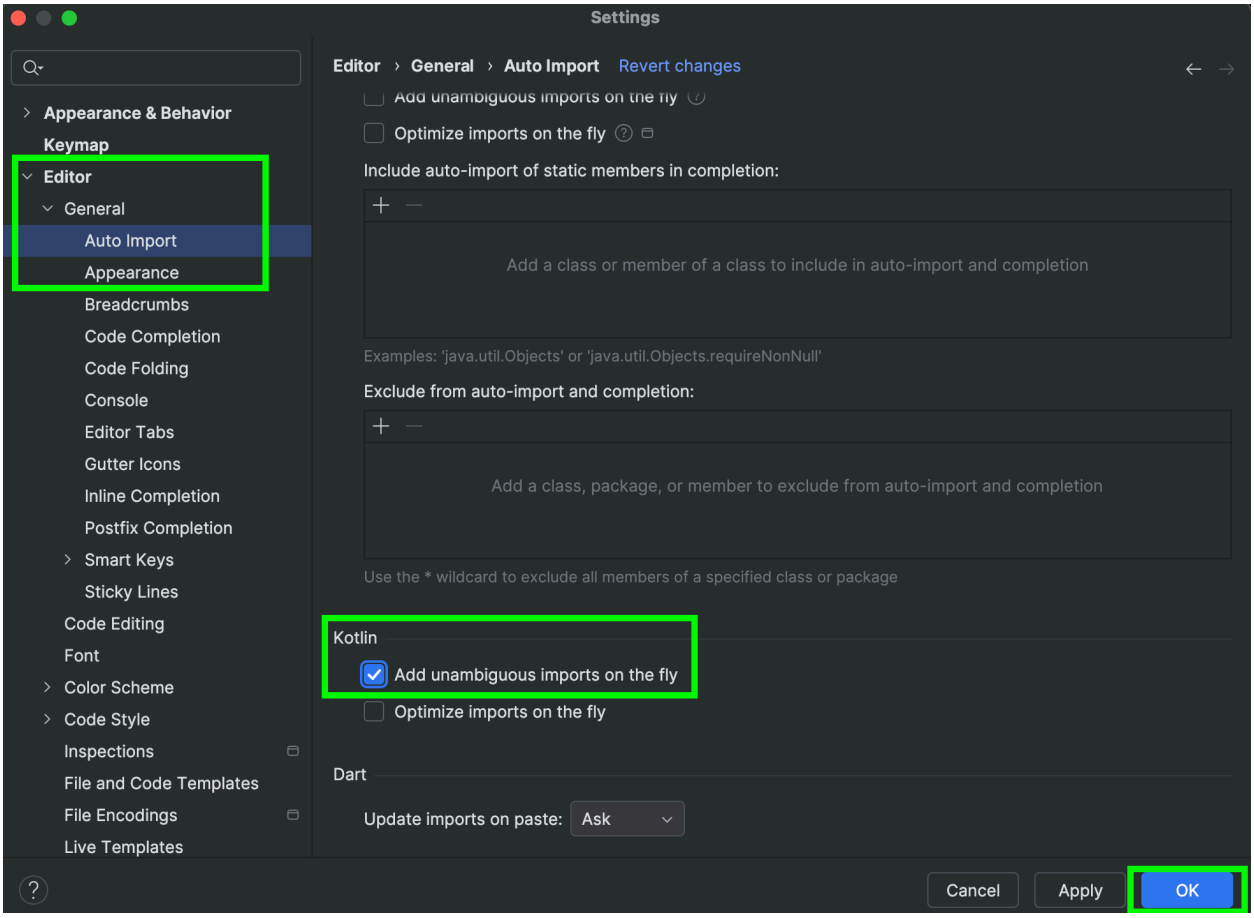
Library Import Guidelines

When you add new code to a project, you will often notice that many import statements are required. It can take some time to get used to this extra step. To help, this section discusses guidelines you can take when selecting imports for each library.

Enabling Automatic Library Imports

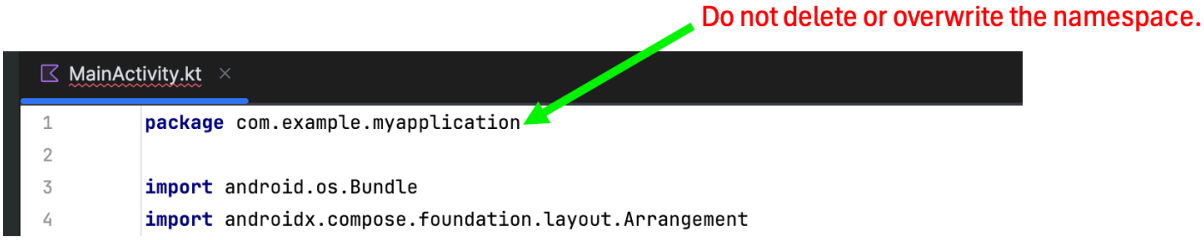
Android Studio can be configured to automatically add any **unambiguous imports**. Unambiguous imports are imports that can be resolved to only one class or function. However, for items like `Text`, which exist in multiple libraries, Android Studio will ask you to choose the correct one manually.

To set Android Studio up for automatic imports wherever possible, go to **Android Studio | Settings**. Then from the **Editor** tab, choose **General** and then **Auto Import**. From there make sure **Add unambiguous imports on the fly** is checked and then click OK.



Do not delete or overwrite the namespace in Kotlin

Try to remember not to delete or overwrite the namespace in Kotlin. This namespace is created when your project is created and the namespace is referenced in more than one area. The project may stop working if the namespace is changed.



Use the Composable Import Whenever Possible

When finding the correct import for an item which is offered by more than one library, try to find the Composable version.

```
35        ) {
36            Text(
```

Imports

- @Composable Text(text: String, ...) (androidx.compose.material3) — Gradle:
- Text (of org.w3c.dom)
- Text (in androidx.compose.foundation.content.MediaType.Companion) — Gradle: an
- Text (in androidx.compose.ui.input.pointer.PointerIcon.Companion)
- Text (in androidx.compose.ui.semantics.SemanticsProperties)
- Text (in androidx.compose.ui.text.input.KeyboardType.Companion)

In the case above, selecting @Composable adds the import statement shown below.

```
package com.example.myapplication

import android.os.Bundle
import androidx.compose.foundation.layout.Arrangement
import androidx.compose.foundation.layout.Column
import androidx.compose.material3.OutlinedTextField
import androidx.compose.material3.Text
```

Here is another example where a composable import is selected for Modifier.

```
// Step 2: Decl
var textState b
var message by

// Step 3: Build the UI layout
Column(
    modifier = Modifier
```

- Modifier (androidx.compose.ui) — Gradle: andro
- Modifier (of java.lang.reflect) — < Android A
- Modifier in WatchEvent (of java.nio.file) — < Android A

Leave Unresolved Objects Alone

If no import suggestions appear when hovering over certain objects, leave them as is. These issues usually resolve automatically once additional imports are added elsewhere in the file.

```
OutlinedTextField(
    value = textState,
    onValueChange = { textState = it },
    label = { Text( text = "Name") },
    modifier = Modifier.fillMaxWidt
)
```

Unresolved reference 'it'.

Create local variable 'it'

Do Not Create Anything

When searching for required libraries do not choose to create anything. The library should already exist.

```
override fun onCreate(savedInstanceState: Bundle?) {
    super.onCreate(savedInstanceState)
    setContent
        MyComp
    }
}
}
```

Unresolved reference 'onCreate'.

Create extension function 'ComponentActivity.onCreate'

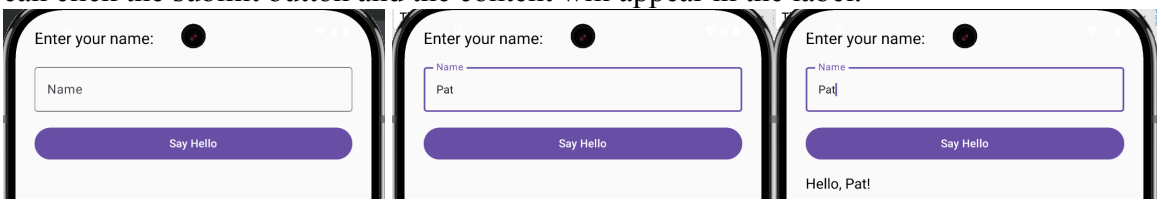
Sometimes Trial and Error is Necessary

After you have imported all composable libraries where available, you may have some straggler objects which still need imports that are not composable. Trial and error should be able to resolve these imports without too much trouble.

Compose: Text Inputs and Buttons

Example 5: Showing Text Inputs and Buttons

This example demonstrates how to enable an editable text input. After users enter content, they can click the submit button and the content will appear in the label.



To build this example, replace the contents of **MainActivity.kt** with this code. **Remember: Do not delete or overwrite the namespace that was generated for your project.**

```
class MainActivity : ComponentActivity() {
    override fun onCreate(savedInstanceState: Bundle?) {
        super.onCreate(savedInstanceState)
        setContent {
            MyComposeApp()
        }
    }
}

@Composable
fun MyComposeApp() {
    // Step 2: Declare state variables to hold text input.
    // Variables declared with ‘var’ can be changed later.
    var textState by remember { mutableStateOf(TextFieldValue("")) }
    var message by remember { mutableStateOf("") }

    // Step 3: Build the UI layout
    Column(
        modifier = Modifier
            .fillMaxSize()
            .padding(24.dp),
        verticalArrangement = Arrangement.spacedBy(16.dp)
    ) {
        Text(
            text = "Enter your name:",
            fontSize = 20.sp
        )

        // Step 4: Create an OutlinedTextField
        OutlinedTextField(
            value = textState,
            // update the state variable.
            onValueChange = { textState = it }, // ‘it’ is latest input from user
            label = { Text("Name") },
            modifier = Modifier.fillMaxWidth()
        )

        // Step 5: Create a Button that reacts to clicks
        Button(
            onClick = {
                message = if (textState.text.isNotBlank()) {
                    "Hello, ${textState.text}!"
                } else {
                    "Please enter your name."
                }
            },
            modifier = Modifier.fillMaxWidth()
```

Exercise 4 (1 mark)

What is the purpose of the `onCreate()` method in `MainActivity`?

uses setContent { } to display the Jetpack Compose UI by calling our composable functions.

Exercise 5 (1 mark)

What is the @Composable annotation for?

It tells Kotlin "this function creates UI" and must be used on any function that builds user interface with Jetpack Compose.

Exercise 6 (1 mark)

Why do we use mutableStateOf() when declaring textState and message?

to create observable state that automatically triggers UI recomposition when the value changes, allowing the interface to react and update whenever the user types text or clicks the submit button.

Exercise 7 (1 mark)

What does it represent in the line onValueChange = { textState = it }?

It says that the new text value entered by the user in the TextField.

Exercise 8 (1 mark)

Why is Modifier.fillMaxWidth() applied to the OutlinedTextField and Button?

makes the OutlinedTextField and Button expand to use the full width of their parent container.

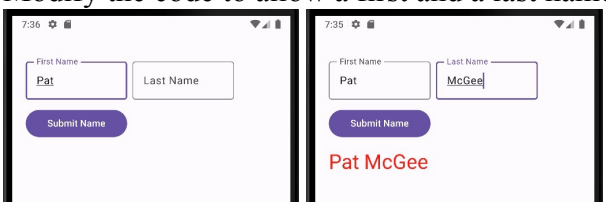
Exercise 9 (1 mark)

What is the difference between declarations that are made with ‘var’ or ‘val’?

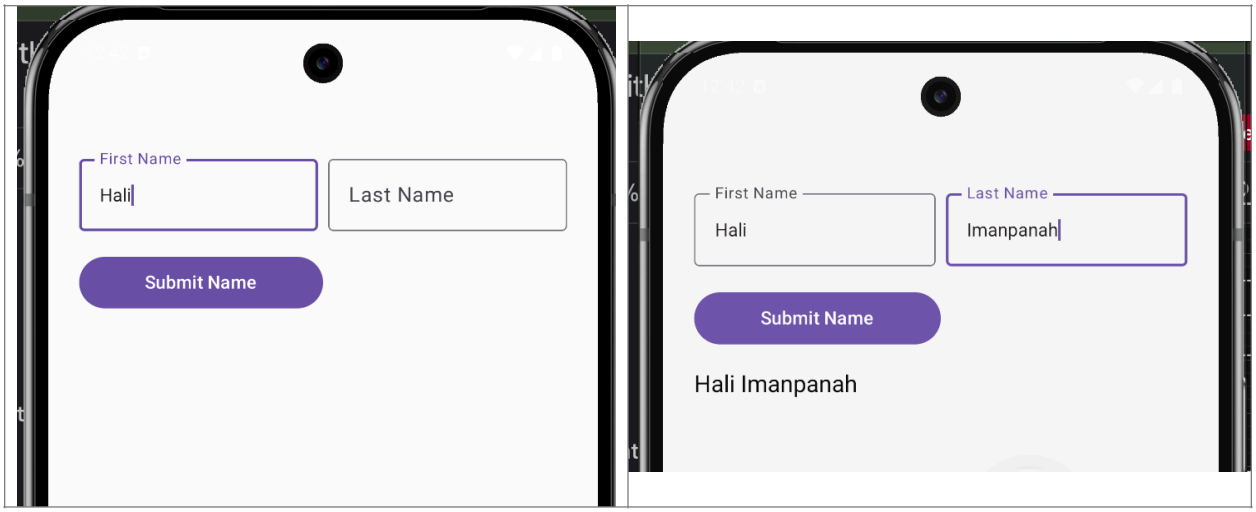
var creates a mutable variable that can be reassigned to different values, while `val` creates an immutable variable that can only be assigned once and cannot be changed.

Exercise 10 (4 marks)

Modify the code to allow a first and a last name.



Show a screenshot after you have made the changes. Enter your name and show the output in the label.



Show all of your MainActivity.kt code as text here (no screenshots please):

```
package com.example.lesso5
```

```
import android.os.Bundle
import androidx.activity.ComponentActivity
import androidx.activity.compose.setContent
import androidx.compose.runtime.getValue
import androidx.compose.runtime.setValue
import androidx.compose.foundation.layout.Arrangement
import androidx.compose.foundation.layout.Column
import androidx.compose.foundation.layout.Row
import androidx.compose.foundation.layout.fillMaxSize
import androidx.compose.foundation.layout.fillMaxWidth
import androidx.compose.foundation.layout.padding
import androidx.compose.material3.Button
import androidx.compose.material3.OutlinedTextField
import androidx.compose.material3.Text
import androidx.compose.runtime.Composable
```

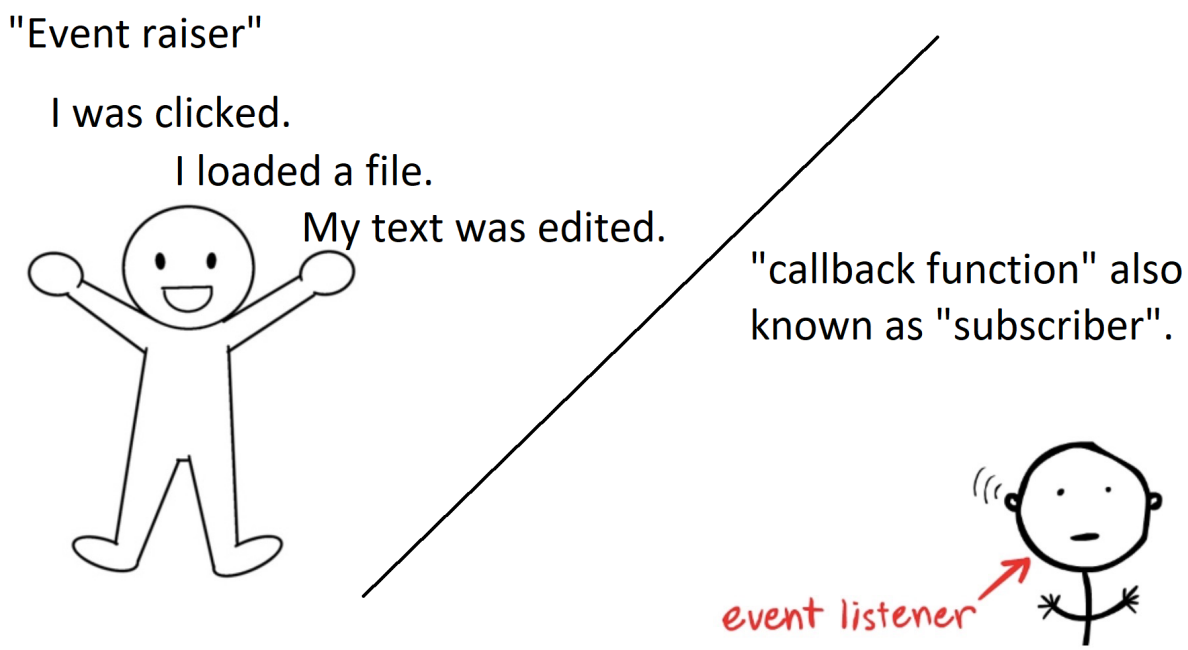
```
import androidx.compose.runtime.mutableStateOf
import androidx.compose.runtime.remember
import androidx.compose.ui.Modifier
import androidx.compose.ui.text.input.TextFieldValue
import androidx.compose.ui.unit.dp
import androidx.compose.ui.unit.sp


class MainActivity : ComponentActivity() {
    override fun onCreate(savedInstanceState: Bundle?) {
        super.onCreate(savedInstanceState)
        setContent {
            MyComposeApp()
        }
    }
}

@Composable
fun MyComposeApp() {
    // Declare state variables for first name and last name
    var firstNameState by remember { mutableStateOf(TextFieldValue("")) }
    var lastNameState by remember { mutableStateOf(TextFieldValue("")) }
    var message by remember { mutableStateOf("") }

    Column(
        modifier = Modifier
            .fillMaxSize()
            .padding(start = 24.dp, top = 100.dp, end = 24.dp, bottom =
24.dp),
```

```
        verticalArrangement = Arrangement.spacedBy(16.dp)
    ) {

        Row(
            modifier = Modifier.fillMaxWidth(),
            horizontalArrangement = Arrangement.spacedBy(8.dp)
        ) {
            // First Name TextField
            OutlinedTextField(
                value = firstNameState,
                onValueChange = { firstNameState = it },
                label = { Text("First Name") },
                modifier = Modifier.weight(1f)
            )

            // Last Name TextField
            OutlinedTextField(
                value = lastNameState,
                onValueChange = { lastNameState = it },
                label = { Text("Last Name") },
                modifier = Modifier.weight(1f)
            )
        }

        // Submit
        Button(
            onClick = {
                message = if (firstNameState.text.isNotBlank() &&
lastNameState.text.isNotBlank()) {
```

```
                    "${firstNameState.text} ${lastNameState.text}"
                } else {
                    "Please enter both names."
                }
            },
            modifier = Modifier.fillMaxWidth(0.5f)
        ) {
            Text("Submit Name")
        }

        Text(
            text = message,
            fontSize = 18.sp
        )
    }
}
```

Handling State with an Event Listener (Callback function)

Usually, an event must occur to change the state of a composable. Composables can set up listeners for these events and they can also issue notifications about their change of state to other listeners.



Example 6: Handling a Custom TextField with Mutable State

In an **Empty Activity** project, add this MyCustomTextField() which encapsulates logic for an OutlinedTextField.

MyCustomTextField.kt

```
@Composable
fun MyCustomTextField(
    strInput: String,
    // void callback function with String parameter.
    textUpdatedCallback: (String) -> Unit, // ->Unit means returns void
) {
    OutlinedTextField(
        value = strInput, // Store and display value.
        onValueChange = {
            // Send an update to subscriber. 'it' is a self-reference to the text.
            textUpdatedCallback(it)
        },
        label    = { Text("Enter some text.") },
        modifier = Modifier.fillMaxWidth()
    )
}
```

Then inside setContent() within the onCreate() function of the MainActivity, add this code:

```
Column(modifier = Modifier.padding(20.dp)) {
    Spacer(modifier = Modifier.height(60.dp))

    // State objects that are defined with ‘remember {mutableStateOf()}’ implies;
    // "When state objects are updated, all objects in the UI referencing it
    // will be updated too."
    var inputString by remember { mutableStateOf(("")) }

    Column(modifier = Modifier.padding(20.dp)) { // Add left margin
        Spacer(Modifier.height(50.dp))
        MyCustomTextField(
            strInput = inputString, // Pass original value to the function.

            // Subscribe to get live updates and handle them within.
            textUpdatedCallback = {

                // Update the state variable when it is modified by the function.
                // 'it' references the value returned from the function.
                inputString = it
            },
        )
        Text(inputString)
    }
}
```

Exercise 11 (2 marks)

When the MyCustomTextField() function is called from MainActivity, what two parameter types are being passed to the function?

String(for the text value) & (string)-> unit (for the callback function).

Exercise 12 (1 mark)

When the MyCustomTextField function sends an updated state value to the calling function is it acting as a subscriber or a publisher?

Publisher

Exercise 13 (1 mark)

When the handler in the MainActivity receives the update in state for the OutlinedTextField is it acting as a publisher or a listener (subscriber)?

listener (subscriber)

Exercise 14 (1 mark)

What part of the user interface is being updated when the state variable in MainActivity is updated?

Text

Exercise 15 (1 mark)

The OutlinedTextField is also receiving event notifications. What notification is it receiving and what callback function is it using to receive the notification?

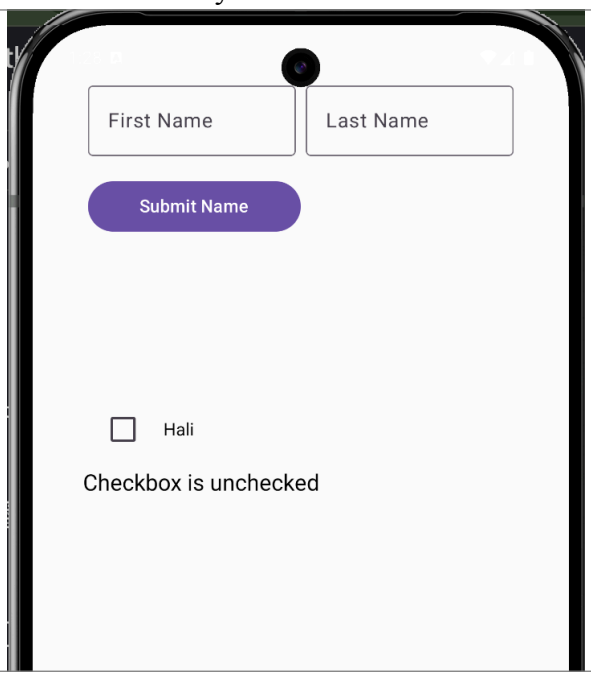
Text change notifications via onValueChange.

Exercise 16 (6 marks)

Create single check box control which is stored in a separate function within a separate file. https://developer.android.com/develop/ui/compose/components/checkbox

Then, in MainActivity figure out how to display the status as to whether or not the checkbox was checked. You will need to pass a listener function from MainActivity to the custom checkbox function to do this.

Show a screenshot of your checkbox with your name beside here:



Show your MainActivity.kt code here for displaying the checked status of the checkbox in a label.

```
package com.example.lesso5
```

```
import android.os.Bundle
import androidx.activity.ComponentActivity
import androidx.activity.compose.setContent
import androidx.compose.runtime.getValue
import androidx.compose.runtime.setValue
import androidx.compose.foundation.layout.Arrangement
import androidx.compose.foundation.layout.Column
import androidx.compose.foundation.layout.Row
import androidx.compose.foundation.layout.Spacer
import androidx.compose.foundation.layout.fillMaxSize
import androidx.compose.foundation.layout.fillMaxWidth
import androidx.compose.foundation.layout.height
import androidx.compose.foundation.layout.padding
import androidx.compose.material3.Button
import androidx.compose.material3.OutlinedTextField
import androidx.compose.material3.Text
import androidx.compose.runtime.Composable
import androidx.compose.runtime.mutableStateOf
import androidx.compose.runtime.remember
import androidx.compose.ui.Modifier
import androidx.compose.ui.text.input.TextFieldValue
import androidx.compose.ui.unit.dp
import androidx.compose.ui.unit.sp
```

```
class MainActivity : ComponentActivity() {
    override fun onCreate(savedInstanceState: Bundle?) {
        super.onCreate(savedInstanceState)
```

```
        setContent {
            MainScreen()
        }
    }
}

@Composable
fun MainScreen() {
    Column(modifier = Modifier.padding(20.dp)) {
        // Name input form
        MyComposeApp()

        Spacer(modifier = Modifier.height(40.dp))

        // Checkbox
        CheckboxSection()
    }
}

@Composable
fun CheckboxSection() {
    var checkboxState by remember { mutableStateOf(false) }

    Column(modifier = Modifier.padding(20.dp)) {
        MyCustomCheckbox(
            checked = checkboxState,
            onCheckedChange = { checkboxState = it },
            label = "Hali"
        )
```

```
        Text(
            text = if (checkboxState) "Checkbox is checked" else "Checkbox is
unchecked",
            fontSize = 18.sp
        )
    }
}

@Composable
fun MyComposeApp() {
    // Declare state variables for first & last name
    var firstNameState by remember { mutableStateOf(TextFieldValue("")) }
    var lastNameState by remember { mutableStateOf(TextFieldValue("")) }
    var message by remember { mutableStateOf("") }

    Column(
        modifier = Modifier
            .fillMaxWidth()
            .padding(start = 24.dp, top = 20.dp, end = 24.dp, bottom =
24.dp),
        verticalArrangement = Arrangement.spacedBy(16.dp)
    ) {
        Row(
            modifier = Modifier.fillMaxWidth(),
            horizontalArrangement = Arrangement.spacedBy(8.dp)
        ) {
            // First Name TextField
            OutlinedTextField(
                value = firstNameState,
```

```
                onValueChange = { firstNameState = it },
                label = { Text("First Name") },
                modifier = Modifier.weight(1f)
            )

            // Last Name TextField
            OutlinedTextField(
                value = lastNameState,
                onValueChange = { lastNameState = it },
                label = { Text("Last Name") },
                modifier = Modifier.weight(1f)
            )
        }

        // Submit
        Button(
            onClick = {
                message = if (firstNameState.text.isNotBlank() &&
lastNameState.text.isNotBlank()) {
                    "${firstNameState.text} ${lastNameState.text}"
                } else {
                    "Please enter both names."
                }
            },
            modifier = Modifier.fillMaxWidth(0.5f)
        ) {
            Text("Submit Name")
        }
```

```
        Text(
            text = message,
            fontSize = 18.sp
        )
    }
}
```

Show your checked box function here.

```
package com.example.lesso5

import androidx.compose.foundation.layout.Row
import androidx.compose.foundation.layout.padding
import androidx.compose.material3.Checkbox
import androidx.compose.material3.Text
import androidx.compose.runtime.Composable
import androidx.compose.ui.Alignment
import androidx.compose.ui.Modifier
import androidx.compose.ui.unit.dp

@Composable
fun MyCustomCheckbox(
    checked: Boolean,
    onCheckedChange: (Boolean) -> Unit,
    label: String
) {
    Row(
        verticalAlignment = Alignment.CenterVertically,
```

```
        modifier = Modifier.padding(8.dp)
    ) {
        Checkbox(
            checked = checked,
            onCheckedChange = onCheckedChange
        )
        Text(
            text = label,
            modifier = Modifier.padding(start = 8.dp)
        )
    }
}
```